St. Cyril and St. Methodius University of Veliko Tarnovo
Faculty of Mathematics and Informatics

Course Project

Subject: Web Programming

Bibliothèque

Students' Names:
KPONOU Primaël
Dovi Kévin-ulcelin

2025/2026 summer semester
(Erasmus+)

Table of Contents

Introduction

Goal

Site map / Architecture

Site Structure

Conclusion

References

Introduction

Bibliothèque is a modern web application designed to provide users with a seamless and visually rich experience for browsing, discovering, and managing digital book collections. The platform connects to the Glose API to fetch real bookshelves and book data, presenting them through an elegant, responsive interface.

Previously, browsing online book catalogs often meant navigating cluttered, outdated interfaces that lacked filtering capabilities and modern design principles. With **Bibliothèque**, we've created a **professional platform to browse book collections, explore detailed book information, search and filter across libraries**, all through a fast, single-page application experience.

Our goal with this platform is not only to display books but also to demonstrate how a modern frontend framework can consume a third-party REST API and deliver a polished user experience with advanced features like real-time search, multi-criteria filtering, pagination, and responsive design across all devices.

Goal

Comparison of different approaches

- **Website builders**: WIX.com; UKIT.com; SITE123.com; WEEBLY.com; JIMDO.com
- **HTML, CSS, JavaScript** (vanilla)
- **CMS** (Content Management System) — WordPress; Drupal; Joomla
- **Modern frontend frameworks** — React (Next.js); Vue.js (Nuxt); Angular; Svelte

	Way 1: Website Builder	**Way 2: Vanilla HTML/CSS/JS**	**Way 3: CMS**	**Way 4: Modern Framework**
Example	Wix, Weebly	Hand-coded HTML	WordPress, Drupal	Next.js, Nuxt.js

Customization	Limited	Full control	Theme-based	Full control
Performance	Slow	Fast if optimized	Medium	Excellent
API Integration	Very limited	Manual fetch	Plugin-dependent	Native support
SEO	Basic	Manual	Good (plugins)	Excellent (SSR)
Scalability	Low	Medium	Medium	High
Learning Curve	Very low	Medium	Low	Higher

Motivation of our choice: Way 4 — Next.js (React)

For the development of **Bibliothèque**, we chose **Next.js 14 with React 18** as our web development framework. The main purpose of our website is to **consume and display data from a third-party REST API** (the Glose API), which requires dynamic data fetching, state management, and component-based architecture — capabilities where modern frameworks excel.

The key requirements that drove our choice:

1. **Dynamic API consumption** — The application needs to fetch bookshelves, book IDs, and individual book details from the Glose API. Next.js with React hooks (`useState`, `useEffect`) provides a clean, declarative way to handle asynchronous data fetching with loading and error states.
2. **Component reusability** — A book card, a search bar, a pagination component — these elements are reused across multiple pages. React's component model allows us to build once and reuse everywhere.
3. **Rich interactivity** — Features like real-time search filtering, sort dropdowns, book detail modals, and responsive mobile menus require JavaScript-heavy interactivity that would be cumbersome with vanilla JS and impossible with a website builder.
4. **Performance** — Next.js provides automatic code splitting, image optimization, and font optimization out of the box.
5. **Modern developer experience** — TypeScript for type safety, Tailwind CSS for rapid styling, and Shadcn/ui components allowed us to build a professional-quality interface efficiently.

Why Next.js over other frameworks?

- **File-based routing** — Creating a new page is as simple as creating a file in the app/ directory.
- **Built-in optimizations** — Font loading, image handling, and automatic code splitting come for free.
- **React ecosystem** — Access to the largest ecosystem of UI libraries, hooks, and community resources.
- **TypeScript first** — Full TypeScript support with strict mode, catching bugs at compile time.

Development Process (Framework vs. Traditional Web Dev)

Unlike traditional web development tools such as **Visual Studio** and **ASP.NET MVC**, which require file structure setup (Controllers, Models, Views), template engines like Razor, manual HTML/CSS/JS integration, and SQL database configurations, Next.js provides a modern, integrated development experience where:

- The **App Router** handles routing automatically based on file structure
- **React components** replace traditional MVC views with a composable model
- **Tailwind CSS** eliminates the need for separate CSS files
- **Custom React hooks** replace controllers with clean, reusable data-fetching logic
- No database is needed — data comes directly from the **Glose REST API**

Key Technologies Used

Technology	Purpose
Next.js 14	React framework with App Router and optimizations
React 18	Component-based UI library
TypeScript 5	Type-safe JavaScript
Tailwind CSS 3	Utility-first CSS framework
Shadcn/ui	49 accessible UI components built on Radix primitives
Lucide React	Icon library

pnpm	Fast, disk-efficient package manager
Jest + Testing Library	Unit and integration testing
Playwright	End-to-end browser testing

Site map / Architecture

The application follows a hierarchical navigation structure centered around the Homepage:

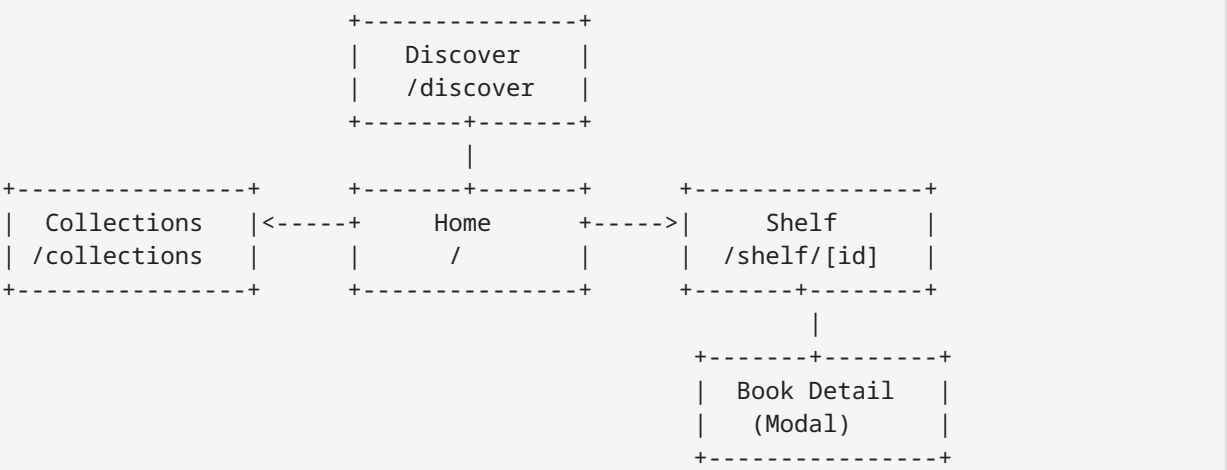
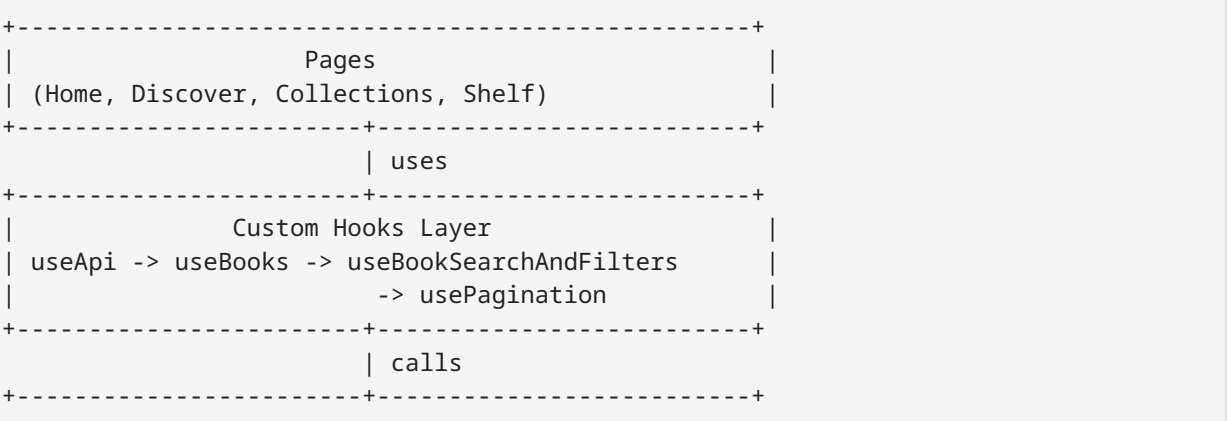


Fig. 1. *Site Architecture*

Data Flow Architecture

The application uses a layered hooks architecture for data management:



```
|             API Client (lib/api.ts)                |
| getBookshelves() | getShelfBooks() |               |
| getBookDetails()                                   |
+------------------------+---------------------------+
                         | fetch
+------------------------+---------------------------+
|            Glose REST API                          |
|            https://api.glose.com                   |
+----------------------------------------------------+
```

Fig. 2. *Data Flow Architecture*

Site Structure

Our website consists of the following pages:

- **Home** (`/`) — Landing page with hero section and bookshelf listing
- **Discover** (`/discover`) — Browse all books across all shelves
- **Collections** (`/collections`) — Overview of all bookshelves with book counts
- **Shelf** (`/shelf/[id]`) — Individual shelf with search and filters
- **Book Detail** (modal) — Detailed book information overlay

Screenshots of different pages

→ “Home”

The homepage welcomes visitors with a full-screen hero section featuring animated gradient backgrounds, platform statistics (10K+ books, 2.5K+ readers), and prominent call-to-action buttons. Below the hero, bookshelves are displayed as interactive cards showing the shelf name, creator, and last modification date.

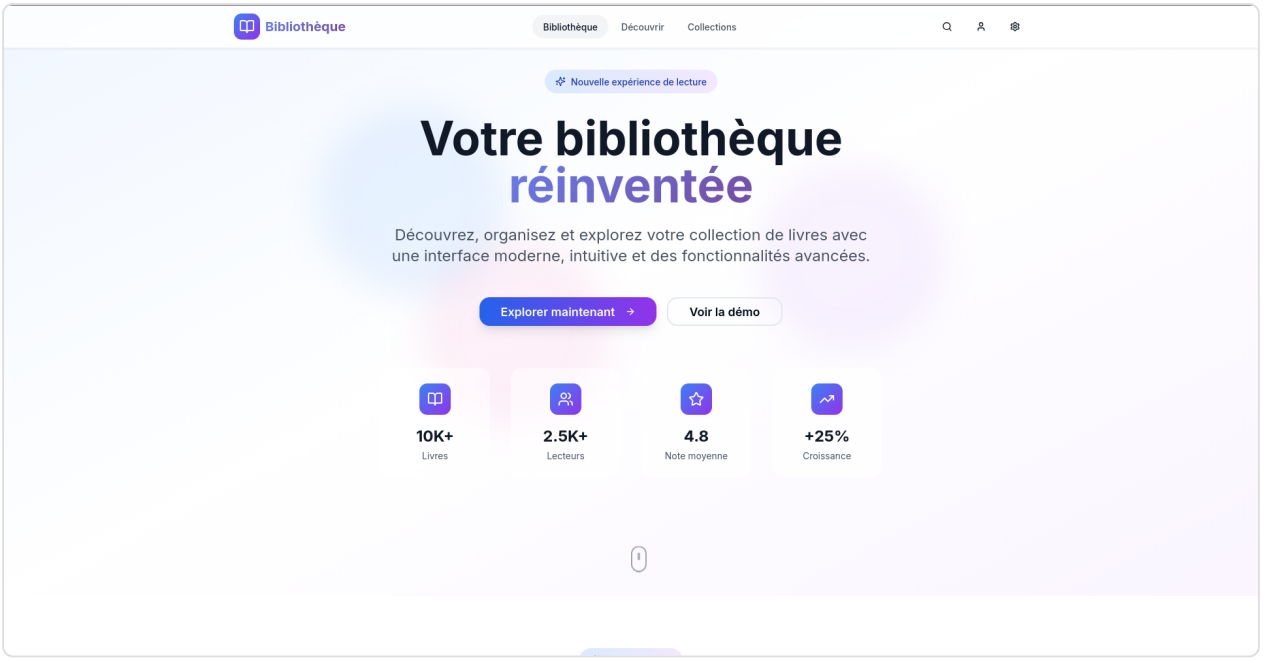


Fig. 3. *Home Page — Hero Section*

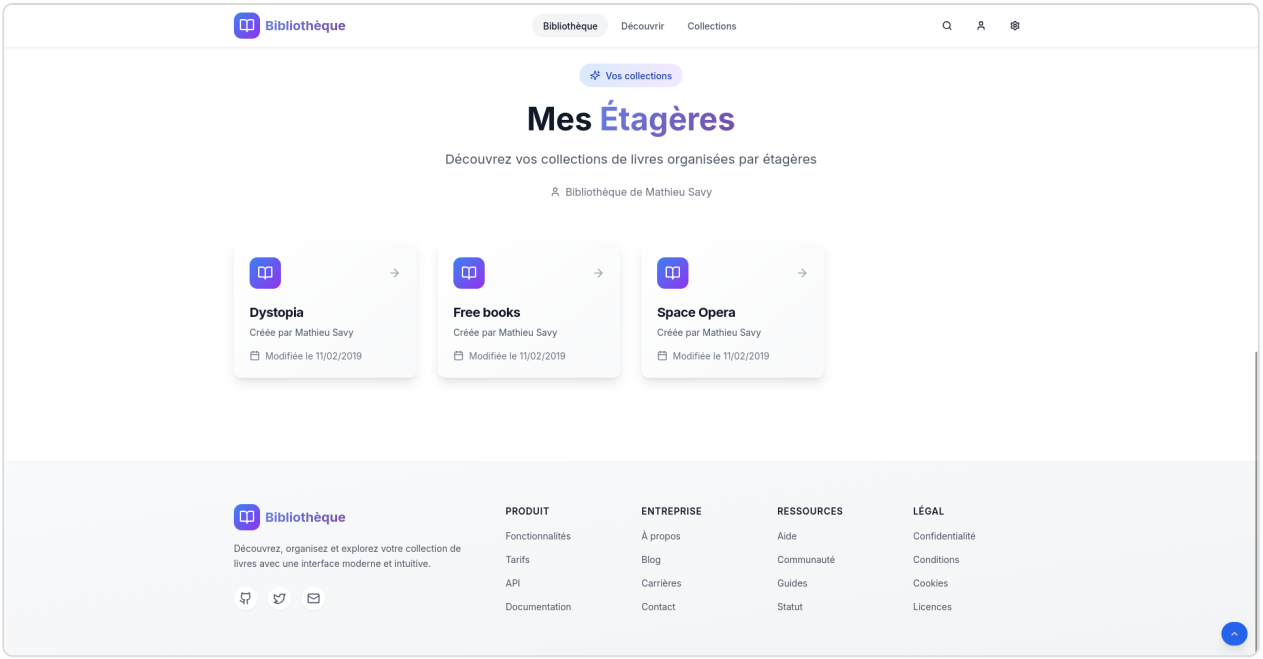


Fig. 4. *Home Page — Shelves Section*

→ “Discover”

The Discover page aggregates all books from every bookshelf into a single browsable view. It features a search bar, sort and filter controls (by title, author, language, free/paid, 18+ content), and results displayed in a responsive card grid with pagination (24 items per

page). Each book card shows the cover image, title, author, language, format, page count, and price badge.

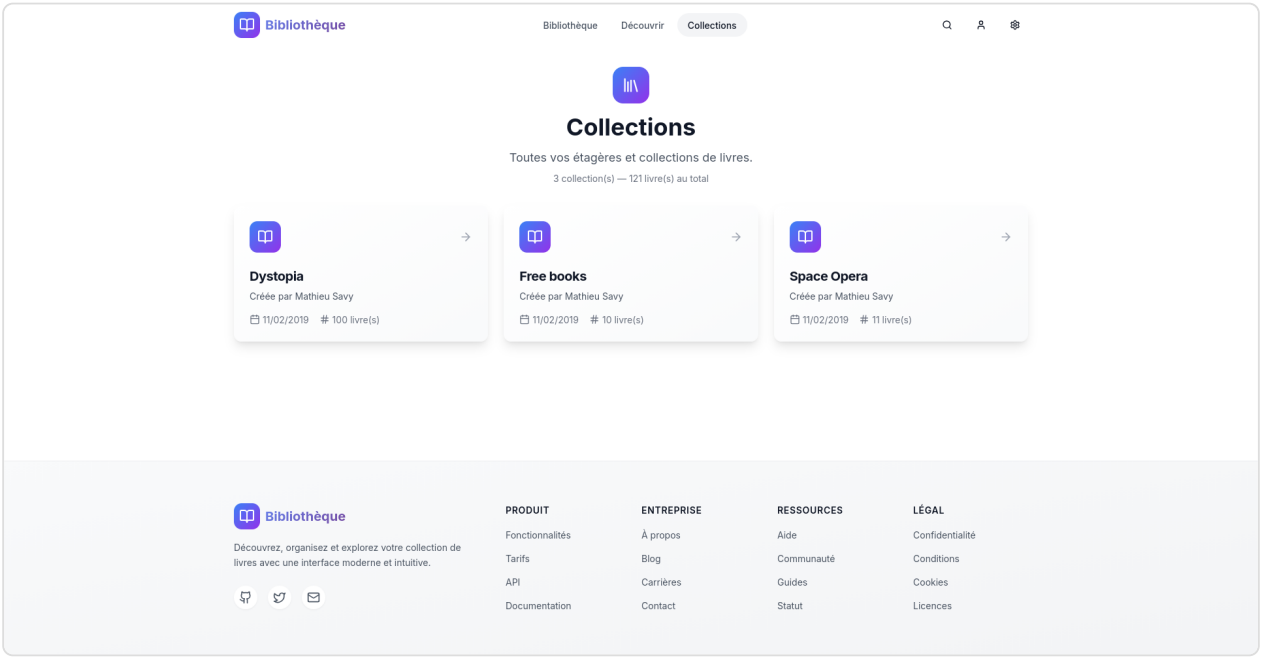


Fig. 5. *Discover Page — Search, Filters & Book Grid*

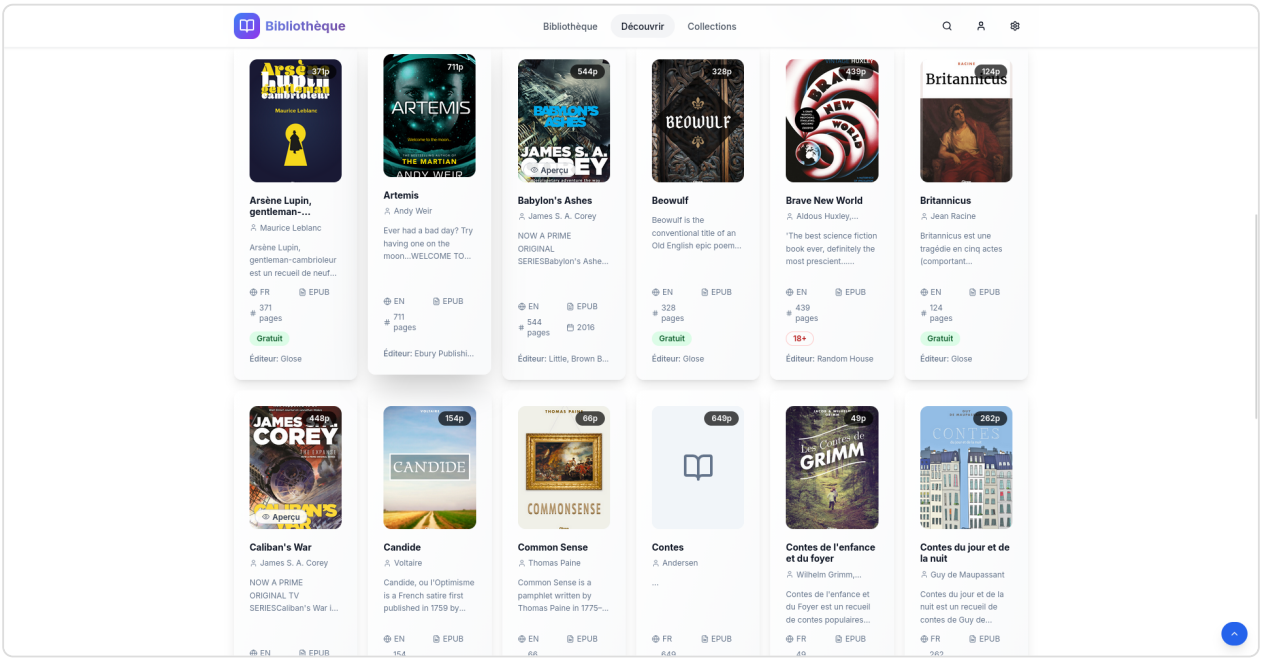


Fig. 6. *Discover Page — Pagination & Footer*

→ “Collections”

The Collections page provides a dedicated overview of all bookshelves. Each collection card displays the shelf name, creator, modification date, and the number of books it contains. The book counts are loaded asynchronously, providing a progressive loading experience.

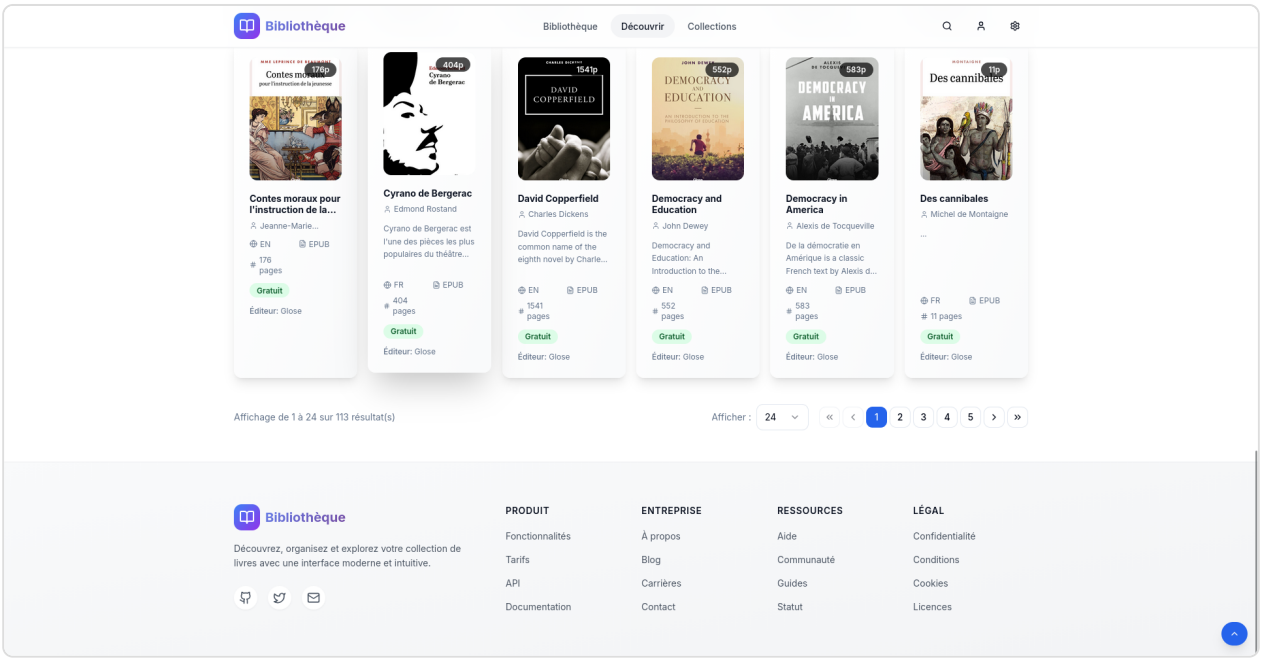


Fig. 7. *Collections Page*

→ “Shelf”

The Shelf page displays all books from a specific bookshelf with a “Back to shelves” navigation button, the total book count, a search bar, and the same filtering/sorting capabilities as the Discover page. Books are loaded in batches of 5 for performance.

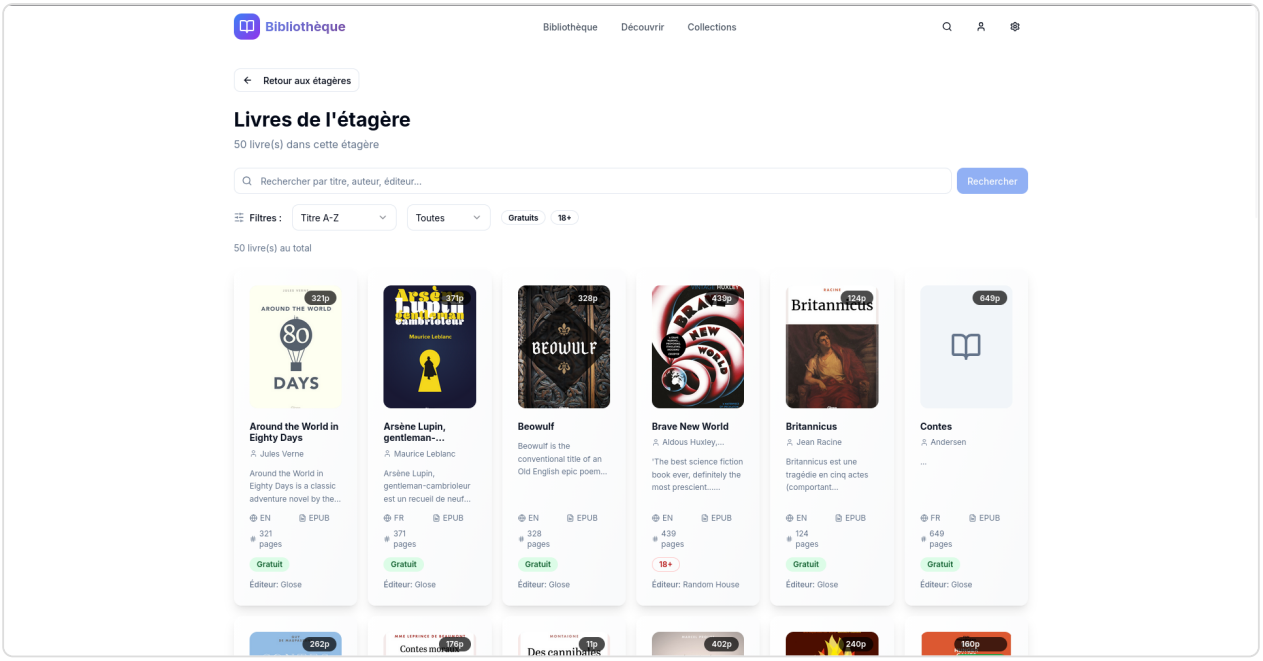


Fig. 8. *Shelf Page — Free Books Collection*

→ “Book Detail” (Modal)

Clicking on any book card opens a detailed modal overlay showing: cover image with page count badge, title, author(s), ISBN, publisher, language, format, page count, and a full description. Action buttons are shown based on the book’s availability.

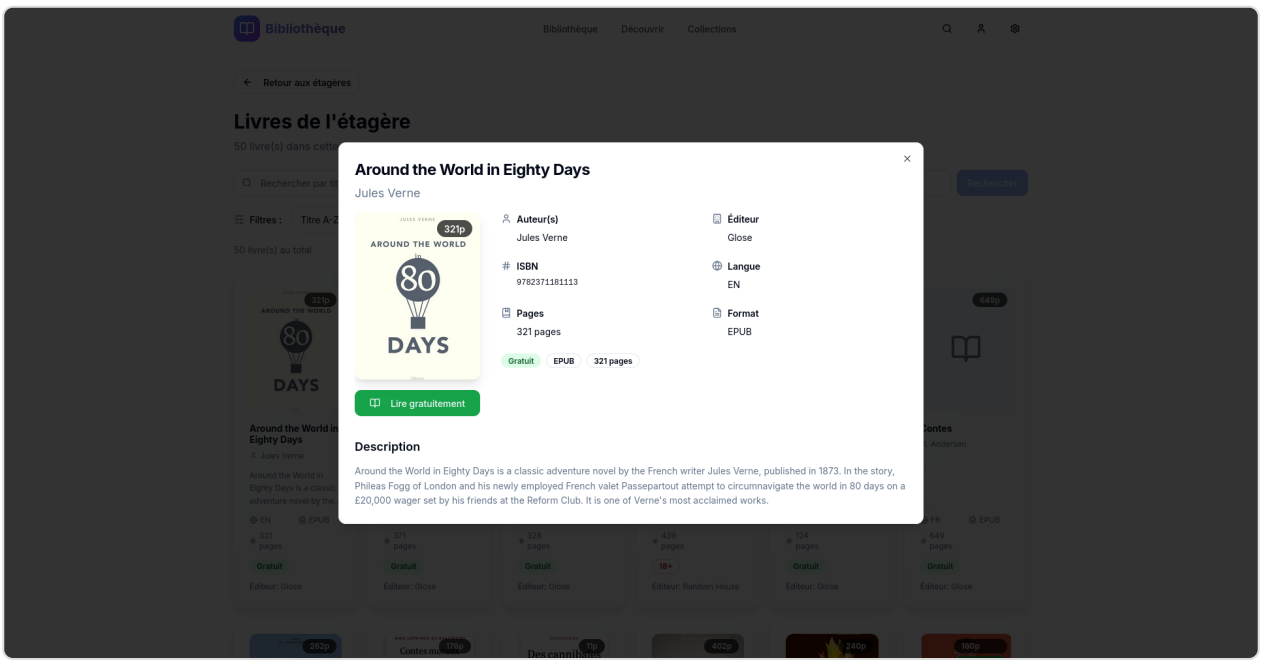


Fig. 9. *Book Detail Modal — “Around the World in Eighty Days”*

→ Mobile Responsive Design

The entire application is fully responsive. On mobile devices, the navigation collapses into a hamburger menu, the book grid switches to a single column, and all interactive elements are touch-friendly.

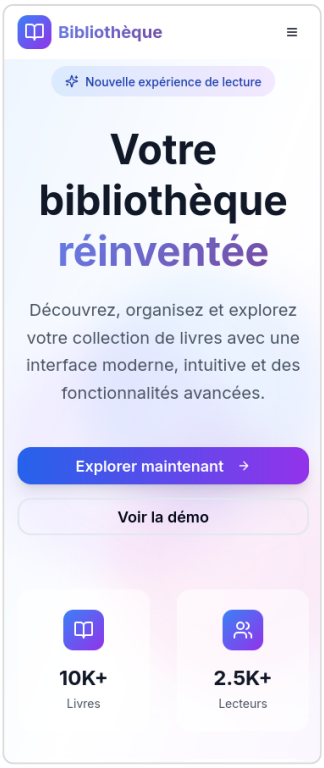


Fig. 10. *Mobile — Hero*

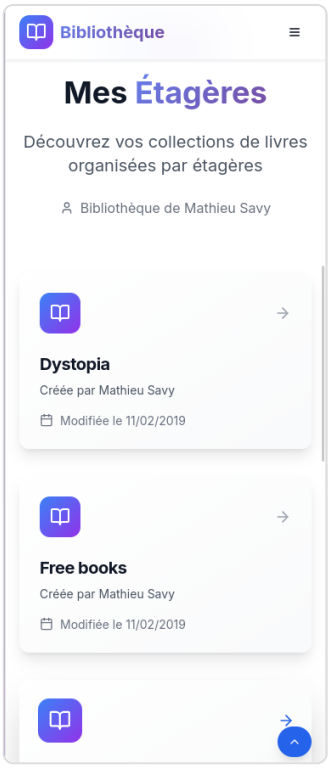


Fig. 11. *Mobile — Shelves*

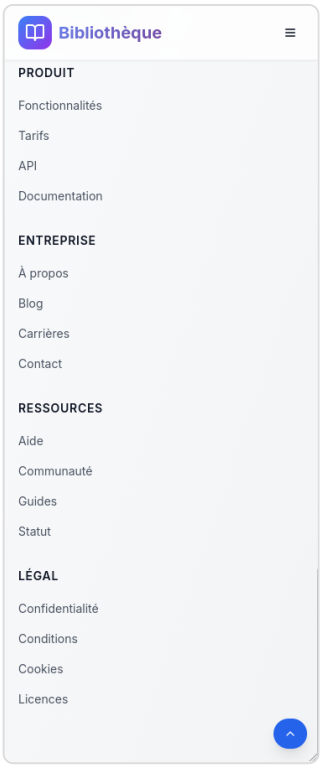


Fig. 12. *Mobile — Footer*

Conclusion

Building **Bibliothèque** with Next.js and React was an excellent learning experience in modern web development. The component-based architecture allowed us to create a complex, feature-rich application while keeping the code organized and maintainable. Custom React hooks provided a clean separation between data logic and presentation.

Key achievements of our project:

- **API integration** — Successfully consumed a third-party REST API (Glose) with proper error handling, loading states, and batch loading for performance.
- **Rich UI/UX** — Built a polished, animated interface with Tailwind CSS and Shadcn/ui components that works seamlessly across desktop, tablet, and mobile devices.

- **Advanced features** — Implemented real-time search, multi-criteria filtering (language, free/paid, adult content), sorting (title, author, page count), and configurable pagination.
- **Testing** — Set up both unit testing (Jest + Testing Library) and end-to-end testing (Playwright) infrastructure, covering components, hooks, and integration flows.

While the learning curve for Next.js and TypeScript is steeper than website builders like Wix, the payoff in terms of performance, flexibility, and code quality is significant. For projects requiring dynamic data fetching, rich interactivity, and a professional user experience, a modern framework like Next.js is clearly the superior choice.

Compared to vanilla HTML/CSS/JS, Next.js saved us considerable development time through component reusability, built-in routing, and the Tailwind CSS utility framework. We estimate the same feature set would have taken 3–4 times longer to build without a framework, with significantly more maintenance burden.

References

1. Next.js Documentation — https://nextjs.org/docs
2. React Documentation — https://react.dev
3. Tailwind CSS Documentation — https://tailwindcss.com/docs
4. Shadcn/ui Component Library — https://ui.shadcn.com
5. Glose API — https://api.glose.com
6. TypeScript Handbook — https://www.typescriptlang.org/docs
7. Radix UI Primitives — https://www.radix-ui.com
8. Playwright Testing Framework — https://playwright.dev